Шаг 1

# Чему учимся

Программа, которую вы запускаете в терминале, работает, пока вы рядом. В этом проекте код уезжает в облако и работает без вас. Его вызывает сайт компании, а он сам ходит в другие сервисы. Так устроена большая часть автоматизаций в компаниях, где между формой, календарём и почтой стоит небольшой код, который их соединяет.

В проекте сходятся темы второго модуля. Это асинхронный код, HTTP-запросы и ответы, JSON, проверка входных данных, даты и часовые пояса, обработка ошибок и тесты.

# Описание проекта

Онлайн-школа проводит бесплатные консультации. Клиент оставляет на сайте заявку, где указывает имя, почту, тему и удобное время. Сейчас заявки разбирает менеджер. Он ищет консультанта по теме, ставит событие в рабочий календарь и сообщает клиенту время. Вы заменяете эту ручную работу одной serverless-функцией в Яндекс Облаке.

Функция принимает заявку и проверяет её. Затем она выбирает консультанта по теме, ставит событие в календарь компании и приглашает на него клиента. Приглашение на почту клиенту рассылает сам календарь. Ответ функции выглядит так:

None

```
$ curl -X POST https://functions.yandexcloud.net/<id> \
    -H 'Content-Type: application/json' \
    -d
'{"name":"Мария","email":"maria@example.com","topic":"frontend
","time":"2026-11-05T12:00:00+03:00"}'
```

{"consultant":"Ирина Ким","start":"2026-11-05T09:00:00.000Z","end":"2026-11-05T10:00:00.000Z"}

Вот как это выглядит в работе. Функция отвечает на заявку, событие с консультантом и клиентом появляется в календаре, а заявки на субботу и с ошибками получают отказ:



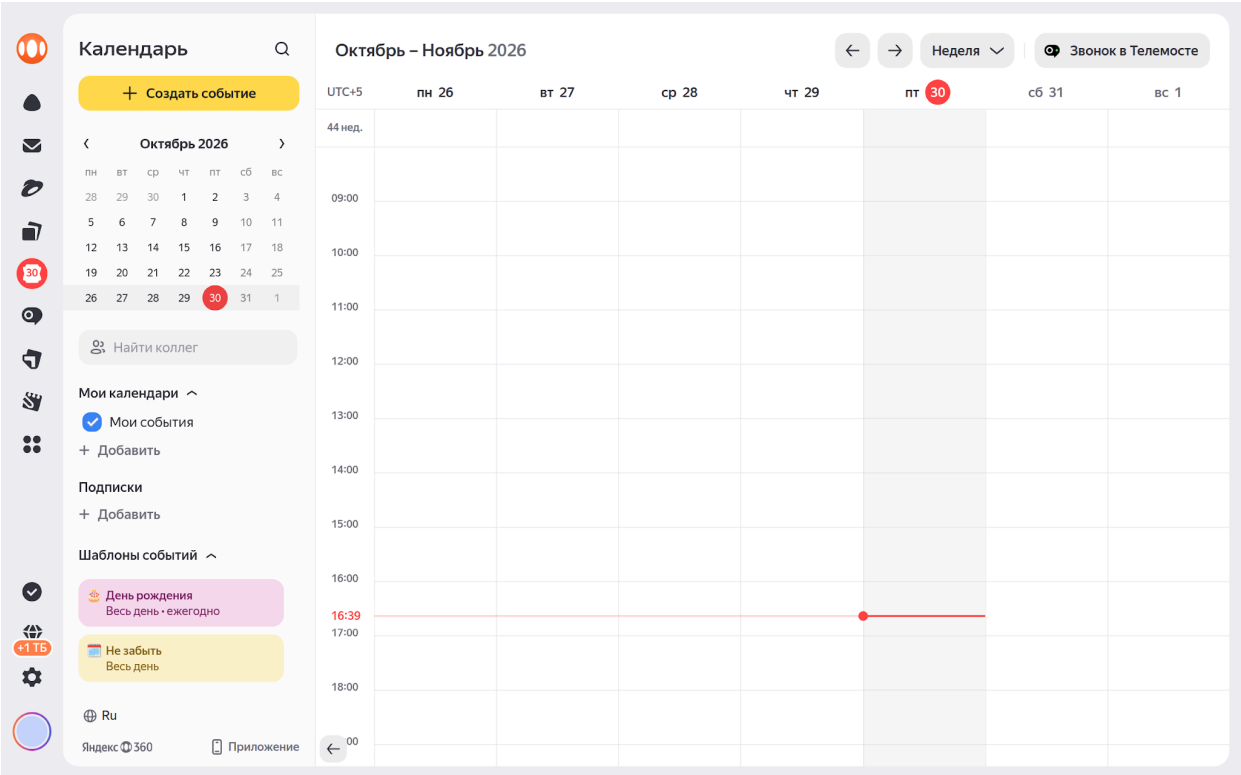
Serverless значит, что сервер вы не настраиваете. Вы отдаёте облаку код, облако запускает его на каждый запрос и берёт плату только за время работы. Код при этом пишется у вас на компьютере и живёт в репозитории, а в облако уезжает одной командой.

# Окружение

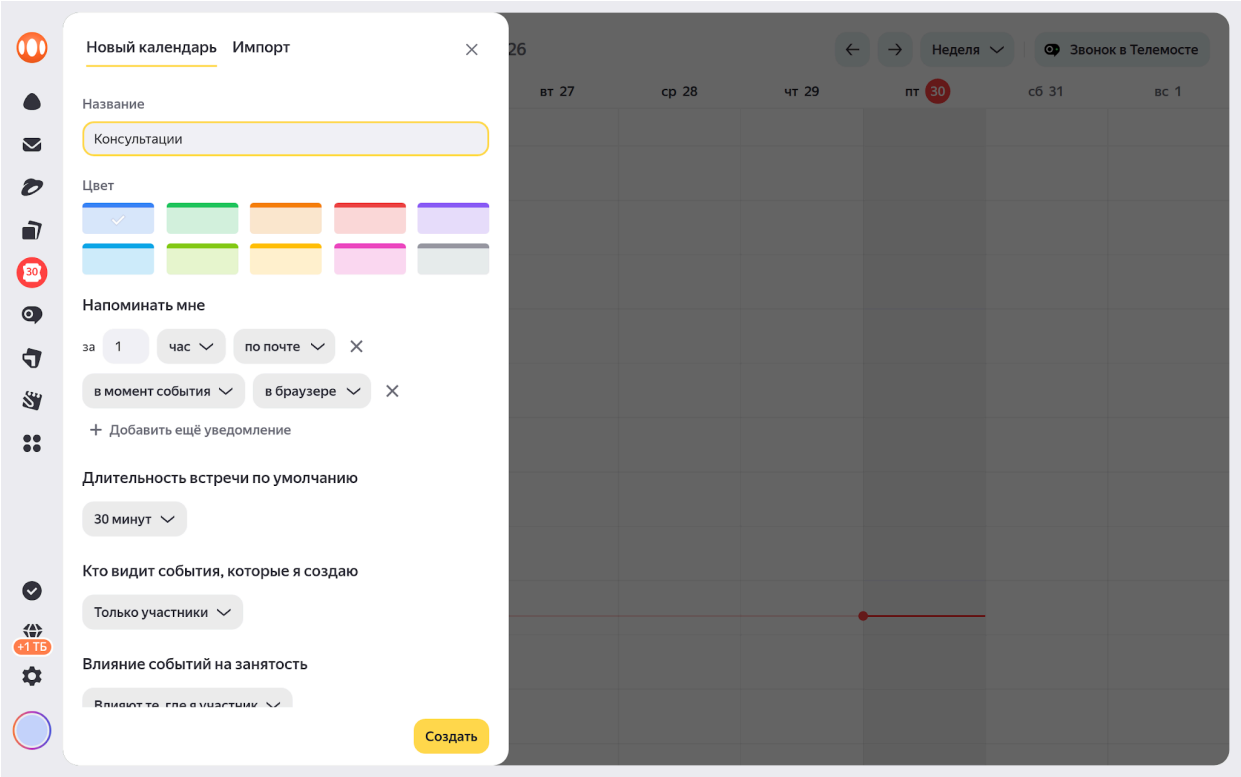
Для проекта нужны три вещи, и все они бесплатны в пределах учебной нагрузки.

1. Аккаунт в Яндекс Облаке с платёжным аккаунтом. Платёжный аккаунт нужен даже для бесплатного уровня, а запросы учебного проекта в бесплатный уровень укладываются.
2. Консольная утилита yc. Через неё функция создаётся и обновляется.
3. Аккаунт Яндекс ID, в котором есть Яндекс Календарь. В этот календарь функция будет ставить события, поэтому для проекта удобно завести отдельный календарь «Консультации».

Отдельный календарь создаётся в Яндекс Календаре. В левой колонке под заголовком «Мои календари» перечислены ваши календари, а под ними стоит кнопка «Добавить».



Кнопка открывает панель «Новый календарь». Впишите в поле «Название» слово «Консультации» и нажмите «Создать» внизу панели. Цвет и напоминания можно оставить как есть, на работу функции они не влияют.




После создания календарь появится в списке «Мои календари». Его адрес для функции вы найдёте в шаге про календарь.

В репозитории уже лежит *consultants.json* со списком консультантов школы и темами, которые каждый из них ведёт. Сайт школы с формой заявки понадобится в последнем шаге.

# Задачи

1. Разберитесь с идеей проекта. На входе заявка с сайта, на выходе событие в календаре и приглашение клиенту.
2. Зарегистрируйтесь в Яндекс Облаке, установите `yc` и выполните `yc init`.
3. Создайте в Яндекс Календаре календарь «Консультации».
4. Склонируйте репозиторий проекта и посмотрите на *consultants.json*.

## Результат шага

Команда `yc config list` печатает ваш каталог в облаке, а репозиторий проекта лежит у вас на компьютере.

## Шаг 2

# Первая функция

Начнём с функции, которая ничего не делает, зато уже работает в облаке. Так вы сразу проходите весь путь от кода в редакторе до адреса, по которому её вызывает кто угодно, и каждый следующий шаг проверяете запросом к живой функции.

Функция в Cloud Functions это обычный модуль JavaScript, который экспортирует асинхронный обработчик. Облако вызывает её на каждый HTTP-запрос и передаёт два аргумента. В `event` лежит запрос (метод, заголовки, тело), в `context` лежат сведения о вызове. Вернуть нужно объект с кодом ответа, заголовками и телом:

None

```
export const handler = async (event) => ({
  statusCode: 200,
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ method: event.httpMethod }),
});
```

Тело ответа всегда строка, поэтому объект перед возвратом превращается в JSON.

# Пакет

Код функции оформляется пакетом Node.js. Облако читает *package.json* и само ставит зависимости из `dependencies`, а `devDependencies` (линтер, тесты) в облако не уезжают. Укажите `"type": "module"`, чтобы работали `import` и `export`.

Обработчик лежит в файле *index.js* в корне репозитория и называется `handler`. Облако найдёт его по точке входа `index.handler`, а автоматическая проверка проекта импортирует `handler` из *index.js*, поэтому имя файла и экспорта менять нельзя.

# Развёртывание

Функция создаётся один раз, а каждое изменение кода превращается в новую версию:

```
None


yc serverless function create --name booking
yc serverless function allow-unauthenticated-invoke booking
```

Вторая команда делает функцию публичной. Без неё вызвать функцию может только тот, у кого есть доступ к вашему облаку, а форма на сайте такого доступа не имеет.

Версия собирается из архива с кодом. В архив идёт только то, что нужно функции, без *node_modules* и тестов:

```
None


deploy:
    rm -f function.zip
    zip -r function.zip index.js package.json
package-lock.json
    yc serverless function version create \
```

```
          --function-name booking \
          --runtime nodejs22 \
          --entrypoint index.handler \
          --memory 128m \
          --execution-timeout 10s \
          --source-path function.zip
```

Адрес функции печатает `yc serverless function get booking` в поле `http_invoke_url`.

# Задачи

1. Оформите пакет, настройте линтер и добавьте в *Makefile* цели `install` и `lint`. Автоматическая проверка ставит зависимости решения командой `npm ci`, поэтому *package-lock.json* должен быть в репозитории.
2. Напишите в *index.js* обработчик, который отвечает кодом 200.
3. Создайте функцию в облаке, сделайте её публичной и добавьте в *Makefile* цель `deploy`.
4. Разверните функцию и вызовите её по адресу.
5. Добавьте *function.zip* в *.gitignore*.

# Требования

- *index.js* экспортирует асинхронную функцию `handler`.
- В *package.json* указан `"type": "module"`.
- В *Makefile* есть цели `install`, `lint` и `deploy`.

# Подсказки

- Рантайм `nodejs22` значит Node.js 22. Проверьте, что у вас локально та же или более новая версия.
- Несколько секунд после `version create` функция отвечает прежней версией. Если ответ не изменился, повторите запрос.

- Если вызов отвечает `502`, загляните в логи функции командой `yc serverless function logs booking`. Чаще всего обработчик вернул объект без `statusCode` или бросил исключение.

## Результат шага

None

```
$ curl https://functions.yandexcloud.net/<id>
{"method":"GET"}
```

## Шаг 3

# Приём заявки

Функцию вызывает чужой код, и данные от него приходят любые. Форма может прислать пустое поле, человек может ошибиться в адресе почты, а кто-то отправит запрос руками и вовсе без тела. Поэтому первое, что делает функция, это проверяет заявку и объясняет, что с ней не так.

Заявка приходит методом POST с телом в JSON:

None

```
{
  "name": "Мария Иванова",
  "email": "maria@example.com",
  "topic": "frontend",
  "time": "2026-11-05T12:00:00+03:00"
}
```

Тело лежит в `event.body` строкой. Иногда облако присылает его закодированным в base64, и тогда `event.isBase64Encoded` равен `true`. Раскодировать такую строку можно через `Buffer.from(body, 'base64')`.

Время записано в формате ISO 8601 и обязательно несёт часовой пояс (`+03:00` или `Z`). Облако работает по UTC, и без пояса «12:00» превратится в 15:00 по Москве.

# Коды ответа

Код ответа говорит вызывающей стороне, что произошло, ещё до того как она прочитает тело. Функция отвечает так.

| Ситуация | Код |
|---|---|
| Метод не POST | 405 |
| Тело не JSON или поля заполнены неверно | 400 |

Тело ответа с ошибкой это JSON с массивом `errors`. Каждая ошибка называет поле, к которому относится, чтобы по ответу было понятно, что исправить:

```
None

$ curl -X POST https://functions.yandexcloud.net/<id> -H
'Content-Type: application/json' -d '{"email":"maria"}'
{"errors":["name: укажите имя","email: укажите адрес

почты","topic: укажите тему","time: укажите время в формате

2026-11-05T12:00:00+03:00"]}
```

Пока заявка проходит проверку, функция может вернуть разобранные данные с кодом 200. Настоящий ответ появится, когда функция научится создавать запись.

# Задачи

1. Прочитайте тело запроса с учётом `isBase64Encoded` и разберите JSON.
2. Проверьте поля `name`, `email`, `topic` и `time`.
3. Верните 405 на любой метод, кроме POST, и 400 со списком ошибок на неверную заявку.
4. Добавьте модуль разбора заявки в архив цели `deploy`, иначе функция в облаке упадёт на `import`. Удобнее держать модули в одном каталоге, например *src/*, и класть в архив весь каталог.
5. Разверните новую версию и проверьте её запросами с верной и неверной заявкой.

# Требования

- Ответ функции с ошибкой содержит JSON вида `{"errors": [...]}`.
- Ошибка неверного поля содержит его имя (`name`, `email`, `topic`, `time`).
- Адрес почты без `@` или без точки в домене считается неверным.
- Время без часового пояса считается неверным.
- Разбор заявки лежит в отдельном модуле, *index.js* его только вызывает.

# Подсказки

- `JSON.parse` бросает исключение на строке, которая не JSON. Поймайте его и превратите в ошибку заявки.
- Формат времени удобно проверить регулярным выражением, а `Date.parse` подтвердит, что такая дата существует.

# Результат шага

Запрос без тела получает код 400 и список из четырёх ошибок, запрос методом GET получает 405.

## Шаг 4

# Выбор консультанта

Менеджер выбирал консультанта по теме заявки. Функция делает то же самое по списку из *consultants.json*, где у каждого консультанта перечислены темы:

```
None
```

```
{
  "name": "Анна Смирнова",
  "email": "anna.smirnova@example.com",
  "topics": ["frontend", "career"]
}
```

Тему часто ведут несколько человек. Тогда функция выбирает одного из подходящих случайно, чтобы заявки распределялись между ними. Если по теме нет никого, записать клиента не к кому, и функция отвечает отказом.

Файл со списком лежит рядом с *index.js* и уезжает в облако вместе с кодом, поэтому его нужно добавить в архив в цели `deploy`. Прочитать его в модуле можно через `readFileSync` и путь от `import.meta.dirname`. Список не меняйте, потому что автоматическая проверка берёт ожидания из него же.

# Отказ по смыслу

Заявка без темы и заявка с темой, которой в школе нет, это разные ошибки. В первом случае данные заполнены неверно, и это 400. Во втором данные верные, но выполнить заявку нельзя. Для такого случая есть код 422 (Unprocessable Content), и тело ответа устроено так же, с массивом `errors`.

# Задачи

1. Напишите функцию выбора консультанта, которая получает список и тему и возвращает одного из подходящих консультантов или `null`.
2. Подключите её в обработчике и верните 422, если консультанта нет.
3. Добавьте *consultants.json* в архив функции и разверните новую версию.

# Требования

- Консультант выбирается только среди тех, у кого тема заявки есть в `topics`.
- Тема без консультантов получает код 422 и массив `errors`.
- Функция выбора не читает файл и не трогает `event`. Она получает список и тему аргументами.

# Подсказки

- Сначала отфильтруйте подходящих, потом выберите случайного через `Math.random()`.
- Передайте функцию получения случайного числа третьим аргументом со значением по умолчанию `Math.random`. В тестах шага про тесты это позволит подставить предсказуемое число.

# Результат шага

Заявка с темой `backend` возвращает консультанта, у которого эта тема есть, а заявка с темой `astrology` получает 422.

# Шаг 5

**В процессе**

# Проверка времени

Клиент выбирает время сам, и выбрать он может что угодно, например вчерашний день, три часа ночи или воскресенье. Функция принимает только то время, когда консультанты работают.

В школе три правила записи.

- консультации проходят по будним дням с 10:00 до 19:00 по Москве;
- консультация длится час, поэтому последняя начинается в 18:00;
- время в прошлом не принимается.

Главная сложность здесь в часовых поясах. Облако считает время по UTC, а рабочие часы заданы по Москве. Время 12:00 по Москве это 09:00 по UTC, и `getHours()` в облаке вернёт 9. В Москве нет перехода на летнее время, смещение всегда +3 часа, поэтому пересчёт сводится к сложению.

Заявка может прийти с любым поясом. `2026-11-05T09:00:00Z` и `2026-11-05T12:00:00+03:00` это один и тот же момент, и функция должна принять оба.

# Ответ функции

Когда время подходит, функция знает уже всё о будущей записи. Теперь она отвечает кодом 201 (Created) и описанием записи:

None

```
{
  "consultant": "Анна Смирнова",
  "start": "2026-11-05T09:00:00.000Z",
  "end": "2026-11-05T10:00:00.000Z"

}
```

Время в ответе записано в UTC через `toISOString()`. Время, которое не подходит, получает 422 с массивом `errors`, как тема без консультантов.

# Задачи

1. Напишите функцию проверки времени, которая получает дату начала и текущий момент и возвращает текст ошибки или `null`.
2. Напишите функцию, которая по началу консультации возвращает её конец.
3. Верните 422 на неподходящее время, а на подходящую заявку 201 с консультантом, началом и концом.
4. Разверните новую версию.

# Требования

- Ответ 201 содержит поля `consultant` (имя консультанта), `start` и `end` в формате ISO.
- `end` на час позже `start`.
- Время вне рабочих часов, в выходной или в прошлом получает 422.
- Текущий момент функция проверки получает аргументом.

# Подсказки

- `getUTCHours()`, `getUTCMinutes()` и `getUTCDay()` дают части даты по UTC независимо от пояса компьютера. Прибавьте к дате три часа и читайте её части через эти методы.

- Сравнивать удобно минуты от начала дня, тогда 18:30 и 19:00 сравниваются одним числом.

## Результат шага

Заявка на будний день в 12:00 по Москве получает 201, а заявка на 09:00 или на субботу получает 422.

# Шаг 6

# Событие в календаре

Запись нужно где-то хранить, и у школы для этого уже есть рабочий календарь. Консультант видит в нём свои консультации, а менеджер видит загрузку. Функция ставит туда событие на каждую запись.

Яндекс Календарь работает по протоколу CalDAV. Это HTTP, в котором календарь похож на папку со своим адресом, а событие похоже на файл в этой папке. Чтобы создать событие, функция отправляет PUT-запрос на адрес вида `<адрес календаря>/<идентификатор>.ics`, а в теле передаёт событие в формате iCalendar:

None

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//Hexlet//Booking//RU
BEGIN:VEVENT
UID:4f1c2d7e-9a3b-4c5d-8e6f-7a8b9c0d1e2f
DTSTAMP:20261104T100000Z
DTSTART:20261105T090000Z
DTEND:20261105T100000Z
```

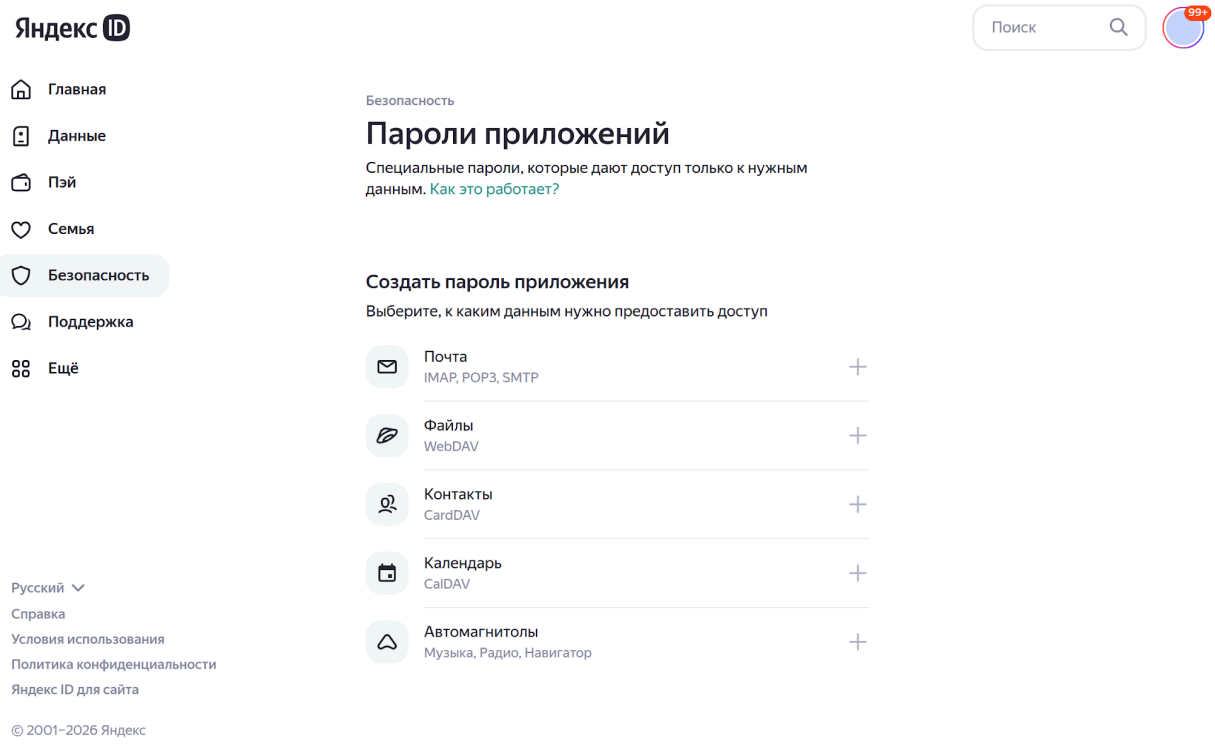
```
SUMMARY:Консультация: Мария Иванова
DESCRIPTION:Тема: frontend. Консультант: Анна Смирнова. Почта
клиента: maria@example.com
ORGANIZER:mailto:school@yandex.ru
ATTENDEE;CN=Анна Смирнова:mailto:anna.smirnova@example.com
ATTENDEE;CN=Мария Иванова:mailto:maria@example.com
END:VEVENT
END:VCALENDAR
```

В формате iCalendar строки разделяются парой символов `\r\n`, а время записывается в UTC без дефисов и двоеточий (`20261105T090000Z`). `UID` у каждого события свой, его удобно получить через `randomUUID()` из `node:crypto` и им же назвать файл. `DTSTAMP` это момент создания события. Строки `ATTENDEE` называют участников, то есть консультанта и клиента, с их почтой.

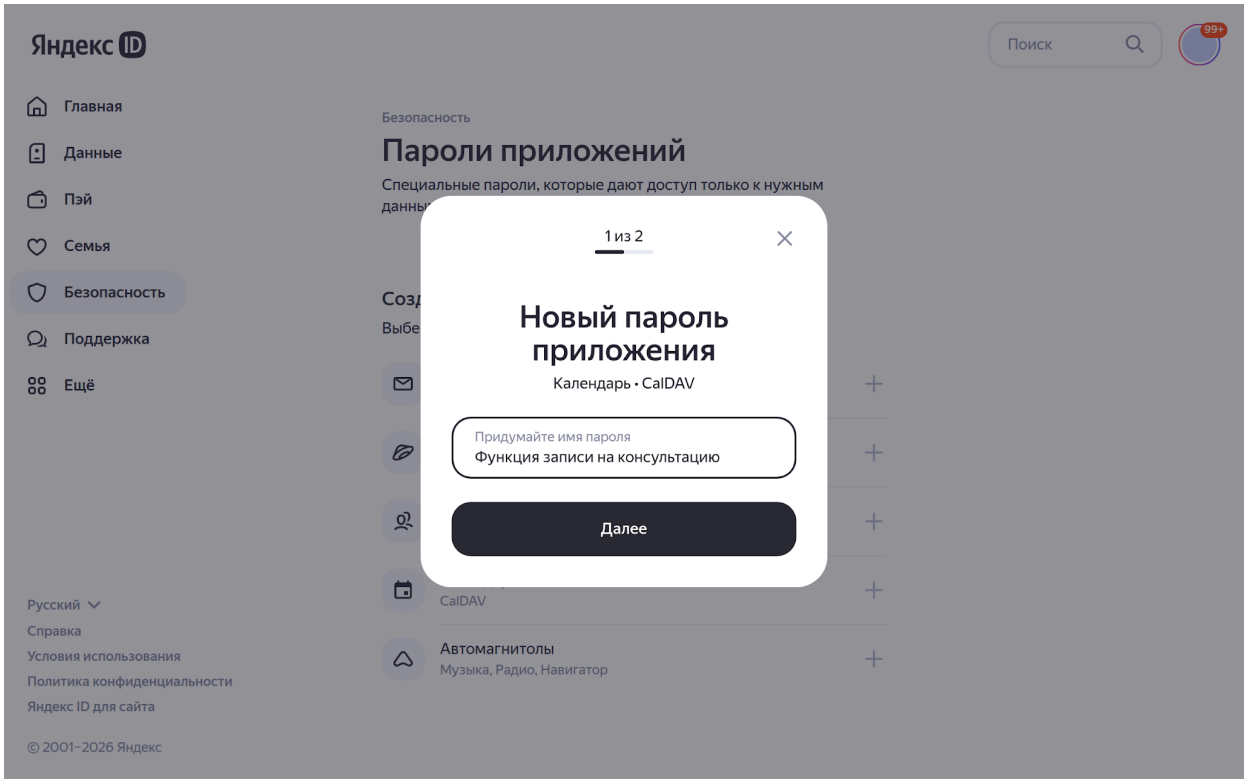
Строка `ORGANIZER` называет владельца календаря, это логин, под которым функция входит в календарь. Без неё Яндекс Календарь молча выбрасывает участников из события. С ней календарь сам рассылает каждому участнику приглашение на почту с временем и кнопками ответа. Так клиент узнаёт о записи, и отдельный почтовый сервис функции не нужен.

# Доступ к календарю

Календарь пускает по логину и паролю приложения. Пароль приложения создаётся в настройках Яндекс ID, в разделе «Безопасность» на странице «Пароли приложений». Ниже списка уже созданных паролей стоит блок «Создать пароль приложения», и каждая строка в нём открывает доступ к одному сервису.




Нажмите на строку «Календарь» с подписью CalDAV. Откроется окно из двух шагов. На первом впишите имя пароля, по нему вы потом отличите этот пароль от остальных, и нажмите «Далее».




На втором шаге Яндекс показывает сам пароль. Скопируйте его сразу, потому что после закрытия окна посмотреть его ещё раз нельзя. Потерянный пароль удаляется в списке на той же странице, и вместо него создаётся новый.

Логин и пароль передаются заголовком `Authorization` по схеме Basic, то есть строкой `логин:пароль` в base64.

Адрес календаря выглядит как `https://caldav.yandex.ru/calendars/<логин>@yandex.ru/events-<номер>/`. Список адресов ваших календарей возвращает запрос `PROPFIND`, в ответе они лежат в тегах `href`:

None

```
curl -X PROPFIND -u "$CALDAV_USER:$CALDAV_PASSWORD" -H 'Depth: 1' \
  "https://caldav.yandex.ru/calendars/$CALDAV_USER/"
```

Здесь `CALDAV_USER` это логин вида `login@yandex.ru`, а `CALDAV_PASSWORD` это пароль приложения.

Пароль в код не пишется. Функция получает адрес календаря, логин и пароль из переменных окружения, а переменные задаются при создании версии:

```
None
```

```
yc serverless function version create ... \
  --environment
CALDAV_URL=$CALDAV_URL,CALDAV_USER=$CALDAV_USER,CALDAV_PASSWOR
D=$CALDAV_PASSWORD
```

Читайте `process.env` внутри обработчика при каждом вызове. Автоматическая проверка поднимает вместо календаря свой сервер и подставляет его адрес в `CALDAV_URL` перед вызовом функции.

# Ошибка сервиса

Календарь может не ответить или ответить ошибкой. Клиенту в этом случае нельзя отвечать 201, ведь записи нет. Для отказа сервиса, к которому функция обращалась сама, есть код 502 (Bad Gateway). Причину отказа функция пишет в лог через `console.error`, а клиенту отвечает общим текстом в `errors`.

# Задачи

1. Напишите функцию, которая по записи собирает текст события в формате iCalendar.
2. Отправьте событие в календарь PUT-запросом с заголовками `Authorization` и `Content-Type: text/calendar; charset=utf-8`.
3. Верните 502, если календарь ответил ошибкой или не ответил.
4. Передайте переменные окружения в цели `deploy`, разверните версию и запишитесь на консультацию на свою почту.

## Требования

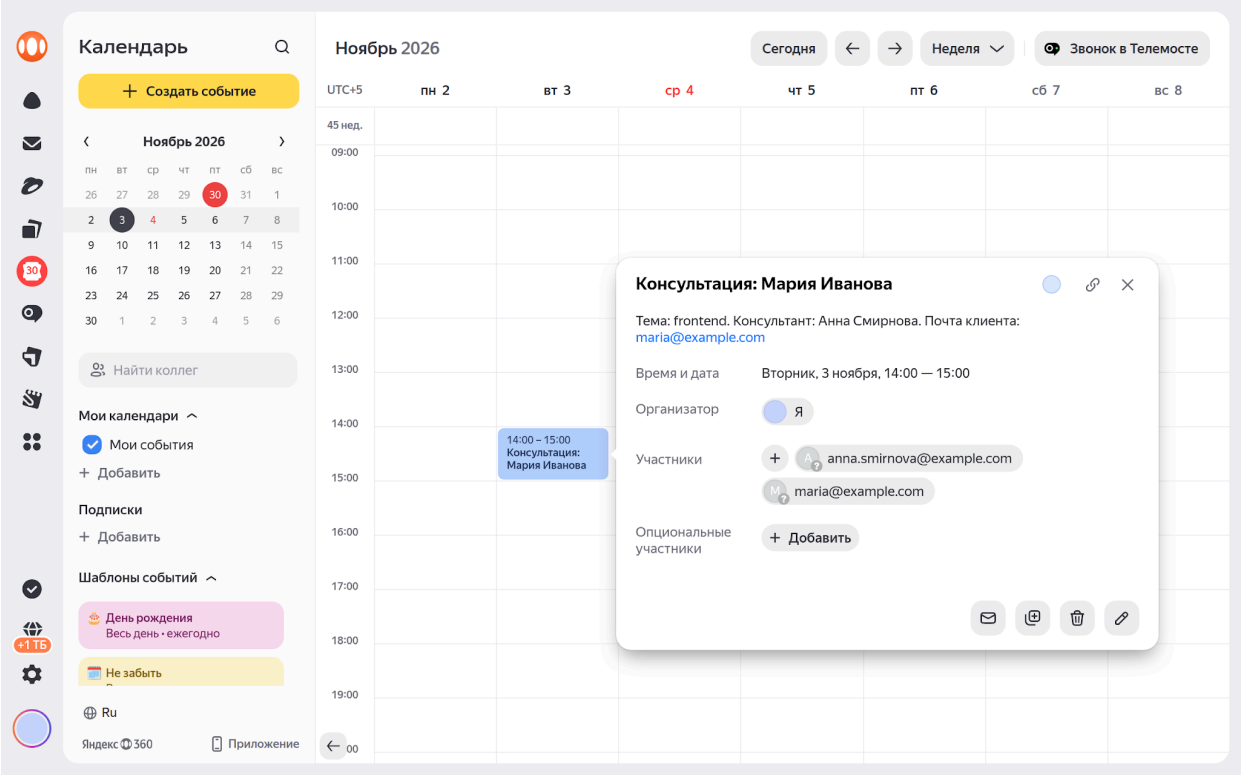
- Функция читает `CALDAV_URL`, `CALDAV_USER` и `CALDAV_PASSWORD` из `process.env` внутри обработчика, при каждом вызове.
- Событие отправляется методом PUT на адрес внутри `CALDAV_URL` с окончанием `.ics`, у каждой записи свой адрес.
- `DTSTART` и `DTEND` события совпадают с началом и концом записи и записаны в UTC.
- Название события (`SUMMARY`) содержит имя клиента.
- Организатор события (`ORGANIZER`) это логин из `CALDAV_USER`.
- Консультант и клиент записаны участниками события (`ATTENDEE`) со своей почтой.
- Отказ календаря даёт ответ 502 с массивом `errors`.

## Подсказки

- `fetch` не бросает исключение на ответ 500, это обычный ответ. Проверьте `response.ok` и бросьте ошибку сами.
- Если сразу после создания пароля приложения календарь отвечает 401, подождите несколько минут.
- Заголовок `If-None-Match: *` просит календарь не перезаписывать событие, если файл с таким именем уже есть.

## Результат шага

После запроса с верной заявкой в Яндекс Календаре появляется событие на выбранное время с именем клиента, а на почту клиента приходит приглашение.




В карточке события видны описание из `DESCRIPTION`, организатор и оба участника из строк `ATTENDEE`. Время календарь показывает в часовом поясе своего владельца. Заявка на 12:00 по Москве на снимке стоит на 14:00, потому что календарь открыт в поясе UTC+5.

# Шаг 7

# Тесты и CI

Каждое изменение до сих пор проверялось развёртыванием и запросом по сети. Это медленно, и проверить так можно только то, что вы догадались отправить. Логика функции при этом лежит в модулях, которые не знают ни про облако, ни про сеть. Выбор консультанта получает список и тему, а проверка времени получает две даты. Такие функции проверяются тестами за доли секунды.

Здесь пригодится то, что случайность и текущий момент передаются аргументами. Тест подставляет `() => 0` вместо `Math.random` и знает, кого выберет функция. Тест передаёт фиксированный «сейчас» и не ломается со временем:

```
None
```

```
test("прошедшее время не подходит", () => {
  const now = new Date("2026-11-04T09:00:00+03:00");

  expect(checkTime(new Date("2026-11-03T12:00:00+03:00"),
now)).not.toBeNull();
});
```

# Задачи

1. Подключите vitest и добавьте в *Makefile* цель `test`.
2. Напишите тесты на выбор консультанта, на проверку времени и на разбор заявки.
3. Настройте CI на GitHub Actions, который ставит зависимости, запускает линтер и тесты. Используйте Node.js 22, как в облаке.

## Требования

- `make test` запускает тесты, и они проходят.
- Тесты проверяют выбор среди нескольких подходящих консультантов, отсутствие консультанта, границы рабочего дня (10:00 и 18:00), выходной и прошедшее время.
- Тесты не ходят в сеть и не зависят от текущей даты.
- CI запускается на каждый push.

## Подсказки

- Файл CI назовите своим именем, например *.github/workflows/ci.yml*. Файл *hexlet-check.yml* принадлежит автоматической проверке Хекслета.
- Граница это место, где ошибка встречается чаще всего. Проверьте 18:00 (последняя консультация) и 18:30 (уже нельзя).

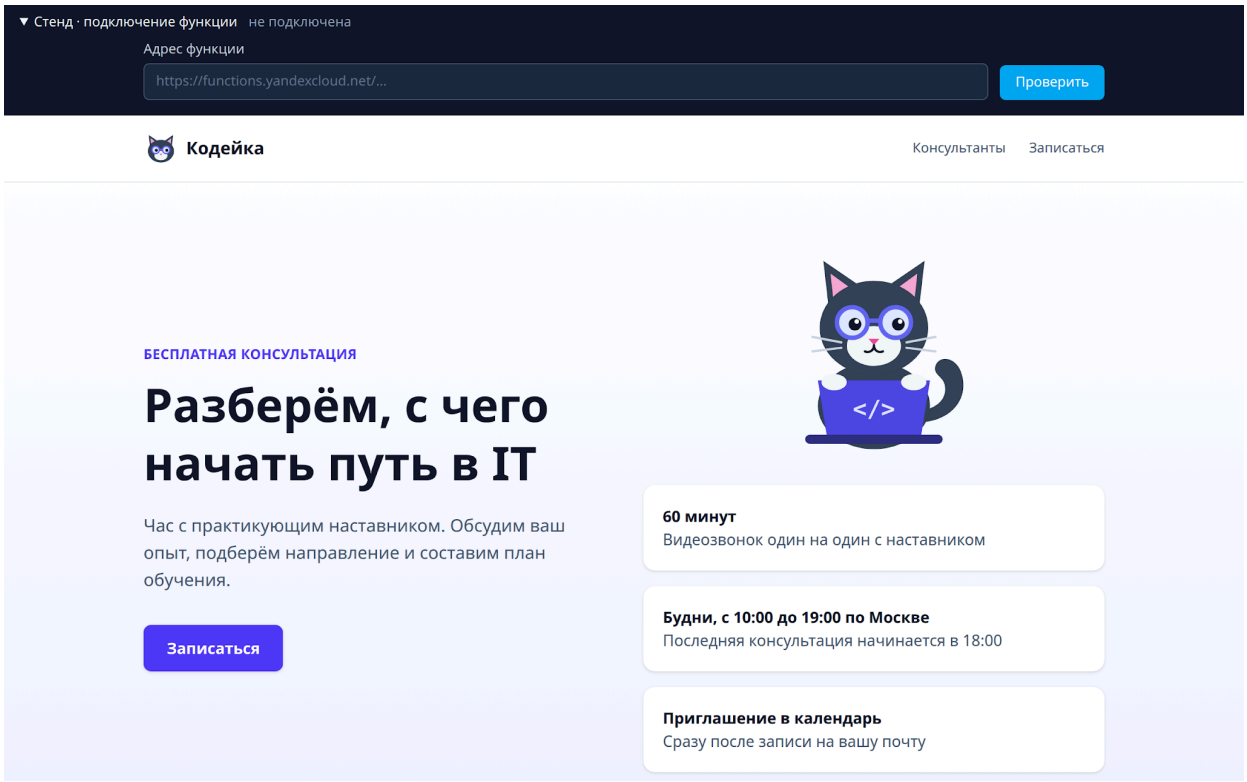
## Результат шага

`make test` проходит локально, а на GitHub рядом с коммитом стоит зелёная отметка вашего CI.

# Шаг 8

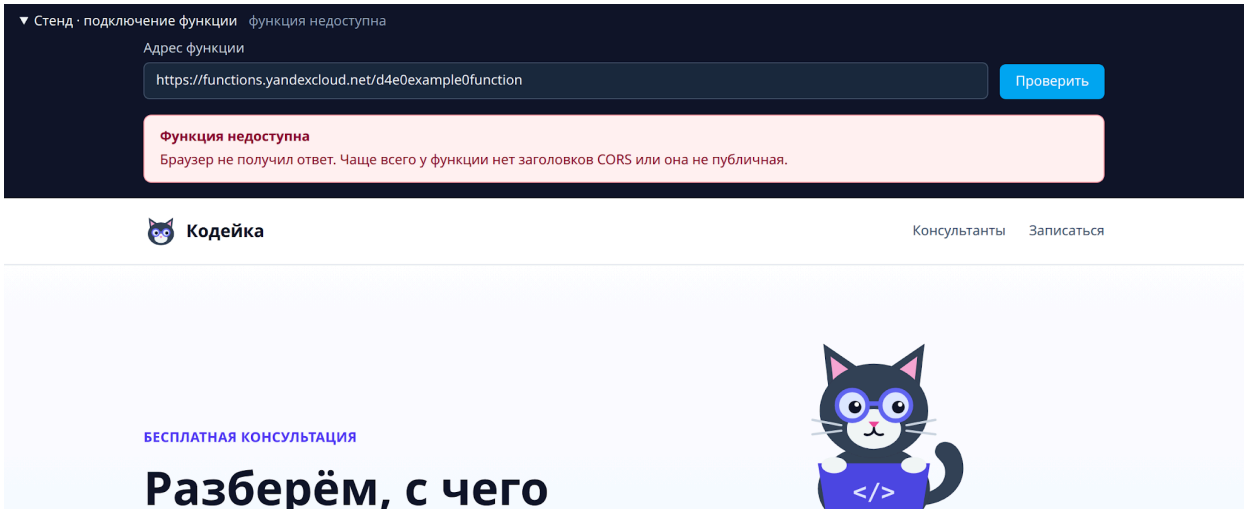
# Форма на сайте

До сих пор заявки отправлял `curl`. В жизни их отправляет форма на сайте, и теперь вы проводите заявку по всей цепочке, от формы до приглашения. Для проекта есть [сайт школы] с формой записи. Вверху страницы тёмная полоса стенда с полем «Адрес функции» и кнопкой «Проверить».




Впишите в поле адрес своей функции и нажмите «Проверить». Стенд отправит пустую заявку и будет ждать ответа 400 с ошибками по полям, как в шаге про приём заявки.

Скорее всего, стенд ответит «Функция недоступна» и подскажет, что у функции нет заголовков CORS.




В консоли браузера при этом появится ошибка про CORS. Сайт открыт с адреса `https://hexlet-components.github.io`, а функция живёт на `https://functions.yandexcloud.net`, и для браузера это разные источники. Браузер не даёт странице с одного адреса читать ответы сервера с другого адреса, пока сервер явно это не разрешит. Перед запросом с телом в JSON браузер отправляет предварительный запрос методом OPTIONS и смотрит на заголовки ответа:

```
None

Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: POST, OPTIONS
Access-Control-Allow-Headers: Content-Type
```

Если заголовков нет, основной запрос браузер не отправляет. `curl` такую проверку не делает, поэтому раньше ошибки не было.

# Задачи

1. Отвечайте на OPTIONS кодом 204 с заголовками CORS.

2. Добавьте `Access-Control-Allow-Origin` во все остальные ответы функции, иначе браузер не покажет странице ни запись, ни ошибку.
3. Разверните версию, проверьте подключение на сайте и запишитесь через форму на свою почту.
4. Опишите в *README.md*, что делает функция, какие переменные окружения ей нужны и как её развернуть.

## Требования

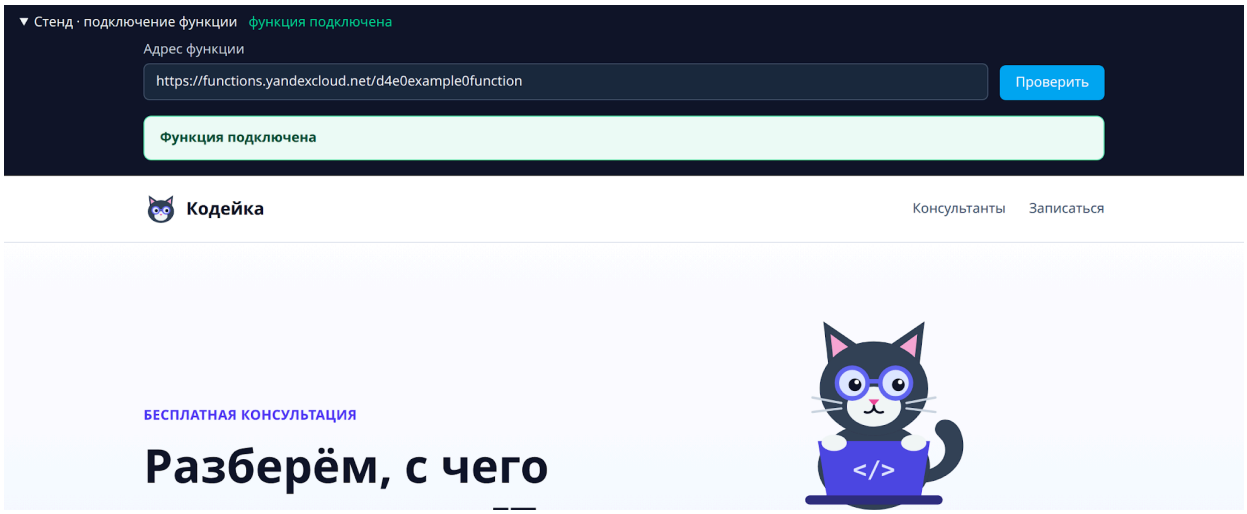
- Запрос OPTIONS получает код 204 и заголовок `Access-Control-Allow-Origin: *`.
- Ответы на POST тоже несут `Access-Control-Allow-Origin`.

## Подсказки

- Сайт запоминает адрес функции в браузере. После новой версии функции достаточно ещё раз нажать «Проверить».
- Заголовки CORS одинаковы для всех ответов, поэтому их удобно держать в одном объекте и добавлять в функции, которая собирает ответ.

## Результат шага

Стенд показывает «Функция подключена».




Заявка из формы создаёт событие в календаре и приглашение клиенту, а сайт показывает консультанта и время записи.